

Projektdokumentation: NYC Collision Analysis

Kurs: DataPy // Prof. Bießmann & Prof. Ipek-Ugay

Projektinformationen

- **Name:** Justus Rothan
- **Matrikelnummer:** 106534
- **Projekttitle:** *NYC Collision Analysis: Streamlit Anwendung zur Analyse von Verkehrsunfällen*
- **Link zum Quellcode:** <https://github.com/TJ-codehub/DataPy>

Kernidee und Datenquelle

Grundidee und Forschungsfragen: Ziel meiner Analyse ist es vorherzusagen, zu welchem Zeitpunkt, sei es Tages- oder Jahreszeit, es am gefährlichsten ist in NYC unterwegs zu sein, basierend auf Unfallstatistiken.

Datenquelle: Als Datenquelle habe ich den „Motor Vehicle Collisions – Crashes“ Datensatz von Data.gov verwendet, der Informationen zu allen von der New Yorker Polizeibehörde (NYPD) erfassten Verkehrsunfällen zwischen 2012 und 2026 enthält.

[<https://catalog.data.gov/dataset/motor-vehicle-collisions-crashes>]

Ich habe mich für diesen Datensatz entschieden, da man Unfallstatistiken gut analysieren und aus ihnen Schlussfolgerungen ziehen kann. Der Datensatz war auch sehr umfangreich, auch wenn viele Spalten oft leer waren, anhand der großen Menge an Zeilen (über 2 Millionen) konnte ich trotzdem viele nützliche Daten verwenden.

Datenbereinigung

Probleme in den Rohdaten: Die unbearbeiteten Daten hatten einige Probleme, die die Verarbeitung erschwert haben:

- Falsche Datentypen (z. B. Datums- und Uhrzeitspalten als reine Text-Objekte).
- Zahlreiche fehlende Werte (NaN / Null) in geografischen Angaben (Breiten-/Längengrade) und Stadtteilen.
- Unpräzise oder nichtssagende Einträge wie 'UNSPECIFIED', 'UNKNOWN' oder leere Zeichenketten in den Spalten für Unfallursachen und Fahrzeugtypen.
- Doppelte Werte bei Fahrzeugtypen („TAXI“ vs. „Taxi“)

Bereinigungsschritte:

1. **Typkonvertierung:** Die Spalte CRASH DATE wurde in ein echtes datetime64-Format transformiert. Textspalten wurden zur Performance-Optimierung explizit in den Pandas-Datentyp string überführt.
2. **Neue Features:** Um zeitliche Analysen zu ermöglichen, wurden neue Merkmale erstellt: CRASH HOUR (Uhrzeit als Stunde) und WEEKDAY (Wochentag als Text).
3. **Filtern ungültiger Werte:** Für geografische Visualisierungen wurden Zeilen ohne valide Koordinaten separiert. Bei der Analyse von Unfallursachen wurden Einträge wie 'UNSPECIFIED' dynamisch herausgefiltert, um Verfälschungen in den Diagrammen zu vermeiden.
4. **Gereinigte Daten:** Nach dem Bereinigen in Pandas blieben über 2,2 Millionen Unfallereignisse (NYC_Collisions_Clean.csv) erhalten, welche als stabile Basis für das Dashboard dienen.

Datenvisualisierung

Ich habe verschiedene Diagramme verwendet, um die gesammelten Daten anschaulich darzustellen:

- **Barcharts:** Darstellung der Top-20-Unfallursachen und der am häufigsten beteiligten Fahrzeugtypen (z. B. Sedans, SUVs).
- **Gefahren-Matrix (Heatmap):** Eine Kreuztabelle aus *Wochentag* vs. *Uhrzeit*.
- **Kombiniertes Trend-Diagramm:** Ein Balkendiagramm der täglichen Unfälle kombiniert mit einer Trendlinie zur saisonalen Glättung im Jahresverlauf.
- **Langzeittrend:** Ein Diagramm zur monatlichen Unfallentwicklung über mehrere Jahre hinweg.

Besonders geeignete Visualisierungen & Erkenntnisse: Ich finde die Wochentag-Uhrzeit-Heatmap ist am aussagekräftigsten. Sie zeigt sehr gut, dass die Unfallzahlen an Werktagen exakt mit den Rush-Hours (08:00 Uhr und 16:00–17:00 Uhr) übereinstimmen. Am Wochenende verlagert sich der Hotspot dann etwas in die Nacht.

Streamlit-Anwendung

Funktionalitäten & Navigation: Die finale Anwendung ist als mehrseitiges Dashboard aufgebaut, welches über eine Sidebar navigiert wird. Sie teilt sich in die Hauptbereiche: *Datenübersicht*, *Data Visualization* (Diagramme und Karte) und *ML Prediction* (*nicht funktional!*) auf.

Interaktive Elemente & Filterlogik: Der Fokus des Dashboards ist der Filterbereich auf der Visualisierungsseite. Nutzer können die Diagramme und Karten nach genauen Kriterien filtern:

- Zeitlich: Jahr, Monat, Wochentag und exakte Stunde.
- Kontextuell: Stadtteil (Borough), Haupt-Unfallursache und Anzahl der beteiligten Fahrzeuge.
- Schweregrad: Mindestanzahl an verletzten oder getöteten Personen via Slider.

Um die Performance zu optimieren habe ich auch die Möglichkeit gegeben die für die Diagramme verwendeten Datenmengen einzuschränken, sodass auch bei vielen passenden Zeilen die Anwendung flüssig läuft.

Pages

Datenübersicht

Data Visualization

ML Prediction

Datenübersicht: NYC Motor Vehicle Collisions

Worum geht es in diesem Datensatz?

Der Datensatz enthält Informationen zu allen von der New Yorker Polizeibehörde (NYPD) erfassten Verkehrsunfällen. Ein offizieller Unfallbericht muss immer dann ausgefüllt werden, wenn Personen verletzt oder getötet wurden oder ein Sachschaden von mindestens 1.000 \$ entstanden ist.

Datensatz-Informationen (Gesamtdatensatz)

Gesamtanzahl der erfassten Unfälle

2,262,089

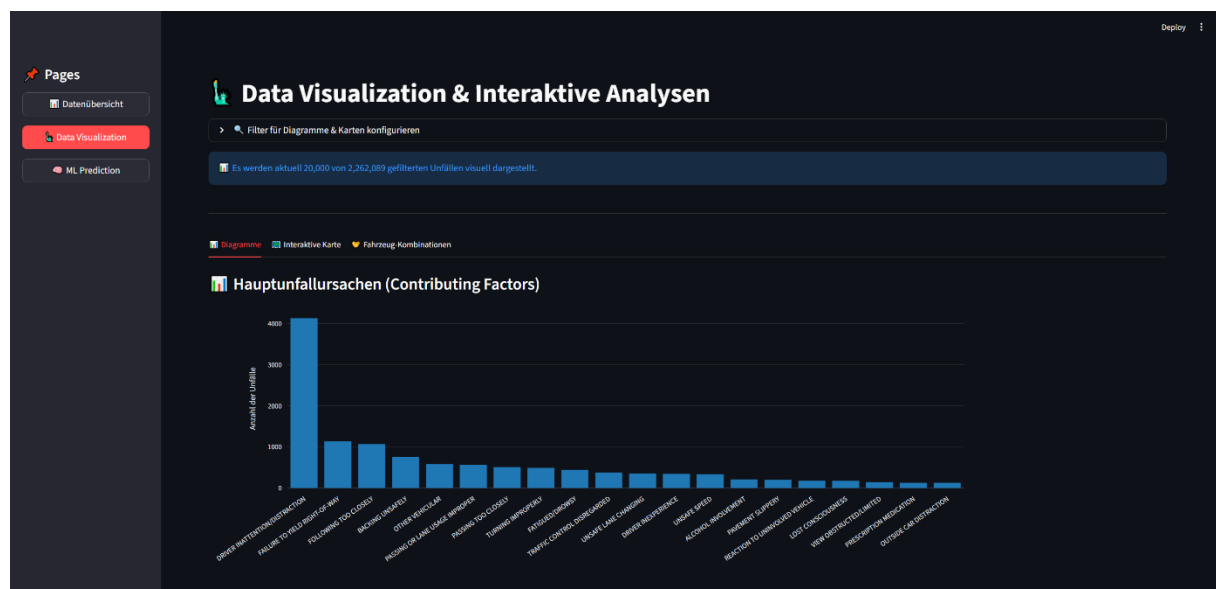
Anzahl der Merkmale (Spalten)

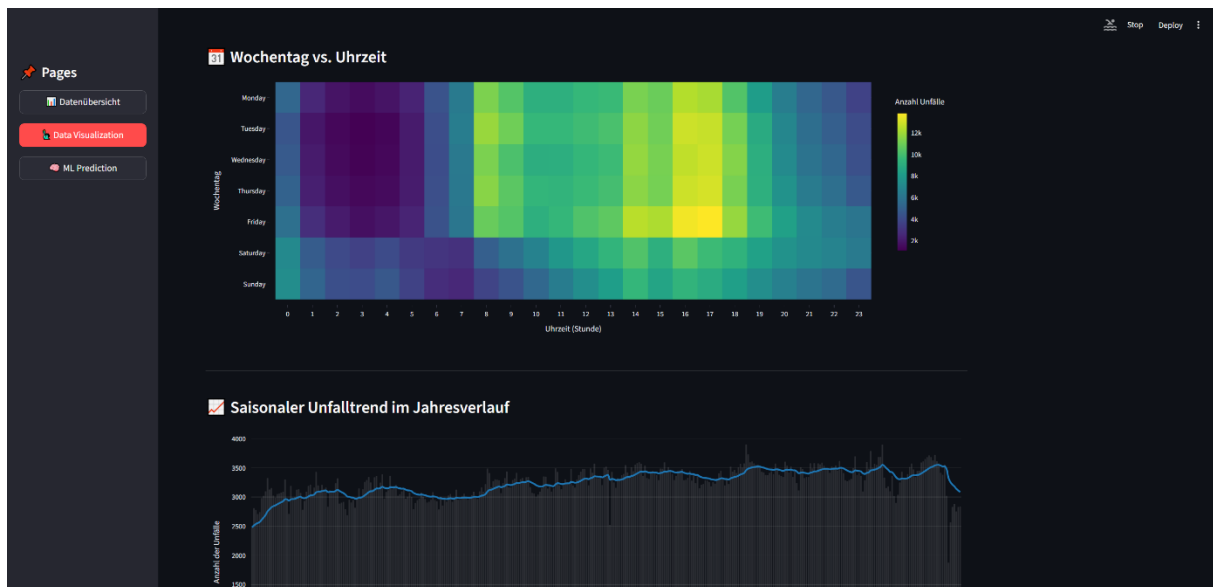
20

Struktur des Datensatzes (Spalten-Details)

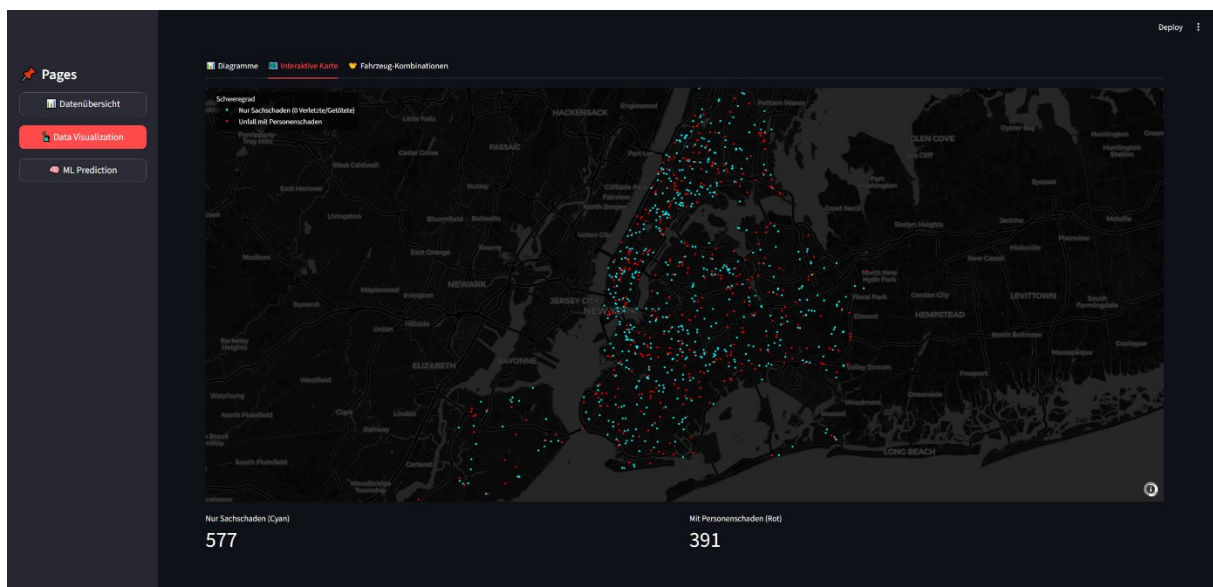
Die folgende Tabelle zeigt alle Merkmale, ihre Datentypen sowie die Anzahl ausgefüllter und fehlender Werte:

Merkmal (Spalte)	Datentyp	Ausgefüllte Werte (Non-Null)	Fehlende Werte (NaN)
CRASH DATE	datetime64[ns]	2262089	0
MONTH	int64	2262089	0
WEEKDAY	string	2262089	0
CRASH HOUR	int64	2262089	0
BOROUGH	string	1572433	689656
ZIP CODE	float64	1572106	689983
LATITUDE	float64	2013757	248332





Visualisierung: Wochentag vs. Uhrzeit Heatmap und Saisonale Unfalltrends.



Visualisierung: Interaktive Karte von NYC mit geplotteten Unfällen.

Reflexion

- **Was hat gut funktioniert?** Das Visualisieren der Daten ist mit verschiedenen Diagramm-Typen gut gelungen. Besonders anschaulich finde ich die Interaktive Karte. Auch konnte ich das Filtern nach mehreren Ansätzen gut und einheitlich über die Anwendung integrieren.

Das Bauen der App hat dank KI relativ gut funktioniert. Nachdem ich die Möglichkeiten von Bibliotheken wie matplotlib/ plotly und streamlit erkannt hatte, konnte ich meine Anforderungen gut kommunizieren.

- **Herausforderungen:** Es gab viele falsch eingetragene Daten, wodurch ich auch nach meinem ersten Data-cleaning immer wieder zurückschauen und meine Pipeline anpassen musste, da sie mir erst bei der Verwendung der App aufgefallen sind.

Auch sind einige Diagramme nicht so aussagekräftig wie ich es am Anfang vermutet hätte. Der Saisonale Verlauf bleibt bei vielen Daten überraschend flach, ich dachte die Jahreszeit hätte einen größeren Impact auf die Unfallmenge. Auch ist die Karte ziemlich Uniform. Ich dachte es gäbe mehr Unfälle an Hauptverkehrsknotenpunkten, auch wenn Hauptstraßen den Großteil ausmachen, sind Unfälle doch sehr gleichmäßig über die Stadt verteilt.